

First Semester 2026 -2027

Project Assignment (Take Home/Open Book)

Distributed Systems with gRPC Communication and Raft Consensus

This assignment focuses on developing robust distributed systems that leverage:

1. **gRPC** communication for inter-service messaging,
2. **Raft** consensus protocol for distributed consistency, and
3. Domain-specific **Large Language Models (LLMs)** for intelligent assistance.

Students must select one project architecture from **six** options: A to F, and build a distributed prototype. The assignment emphasizes distributed consensus, efficient RPC communication, and integrating specialized AI/LLM models.

Note: It is a collaborative team assignment. The desirable team size is THREE members. Teams with different sizes with valid reason/exceptional situations require explicit approval from the Course Instructor.

General Project Guidelines

Plagiarism Policy

- a. A mandatory plagiarism check will be conducted. Ensure your work is original.
- b. Marks will be deducted based on the extent of plagiarism found.

Creative Scope

- a. The instructions provided are the bare minimum requirements.
- b. Some components are intentionally left open-ended to encourage creativity and individual design, additional meaningful features, execution, and overall quality. Appropriate credit will be given accordingly.

Submission Expectations

- a. The submitted project must follow a proper structure and represent a well-thought-out, legitimate effort.
- b. Random, unstructured, or non-functional submissions will be graded accordingly.

System Architecture Options

Select one of the following architectures for your group project. Each project is split into two milestones for structured delivery.

A. Distributed Ticket Booking System

Milestone 1: Project Foundation, gRPC & LLM Integration	Milestone 2: Raft Consensus & Fault Tolerance
- Real-time seat reservation with concurrency control to prevent race conditions. - Payment processing (mock/stub service acceptable). LLM Integration: Customer support chatbot for FAQs (e.g., "How to cancel a booking?", "What seats are available? Etc.").	- Overbooking prevention through distributed consistency. - Booking state replication using Raft across nodes. - Fault tolerance and recovery on leader/follower failure.

B. Distributed Inventory Management System

Milestone 1: Project Foundation, gRPC & LLM Integration	Milestone 2: Raft Consensus & Fault Tolerance
- Order processing with correct stock decrement, preventing negative stock levels. LLM Integration: Demand prediction based on historical data (mock). LLM Integration: Automated reorder suggestions. LLM Integration: Summarized inventory analytics and reports.	- Multi-warehouse stock tracking with synchronized updates across nodes. - Supplier coordination for restocking, replicated via Raft. - Strong consistency for inventory state across all nodes under concurrent orders. - Fault tolerance and recovery for warehouse node crashes.

C. Distributed Real-time Collaboration Platform

Milestone 1: Project Foundation, gRPC & LLM Integration	Milestone 2: Raft Consensus & Fault Tolerance
• Simultaneous multi-user document editing with conflict resolution. • Real-time update propagation to all active users. • Presence awareness (who is online / editing). • LLM Integration: Grammar correction and auto-suggestions. • LLM Integration: Summarization of collaborative notes. • LLM Integration: Intelligent content enhancement.	• Version history with consistent storage across nodes using Raft. • Reliable recovery of document state after crashes.

D. Distributed Healthcare Record System

Milestone 1: Project Foundation, gRPC & LLM Integration	Milestone 2: Raft Consensus & Fault Tolerance
• Patient data storage and retrieval via gRPC. • Role-based access (doctor, nurse, patient) with secure authentication. • Privacy-preserving storage (masking sensitive identifiers). • LLM Integration: Medical query assistant for diagnosis suggestions. • LLM Integration: Treatment recommendation support. • LLM Integration: Summarization of patient history.	• Multi-hospital data replication using Raft. • Strong consistency for critical medical updates using Raft. • Fault tolerance ensuring records remain available despite node failures.

E. Distributed Real-time Chat and Collaboration Tool

Milestone 1: Project Foundation, gRPC & LLM Integration	Milestone 2: Raft Consensus & Fault Tolerance
• Multi-user messaging in group chats/channels. • File sharing between users. • Administrator privileges (manage users, channels). • Presence management (track online/offline status). • LLM Integration: Smart replies to incoming messages and Conversation summarization. • LLM Integration: Context-aware suggestions.	• Message ordering and reliable delivery using Raft replication. • Fault tolerance and recovery when chat servers fail.

F. Distributed Online Auction System

Milestone 1: Project Foundation, gRPC & LLM Integration	Milestone 2: Raft Consensus & Fault Tolerance
• Real-time bid placement with concurrency control to prevent race conditions. • Automated auction timer management and winner determination. • Payment and escrow processing (mock/stub service acceptable). • LLM Integration: Bidding assistant chatbot for FAQs. • LLM Integration: Automated generation of rich item descriptions. • LLM Integration: Summarization of auction results and historical pricing trends.	• Auction state and bid history replication using Raft across nodes. • Fault tolerance and recovery on leader/follower failure during active auctions.

Technical Requirements

Requirement Area	Specifics
Communication Protocol	• Mandatory: All inter-service communication must utilize gRPC framework • Language: Implementation exclusively in Python
Consensus Mechanism	• Algorithm: Implement Raft Consensus Protocol for distributed coordination • Components: Leader election, log replication, failure detection • Consistency: Ensure strong consistency for critical operations
AI Integration	• Use domain-specific LLMS (CPU-optimized for local deployment). • This shall be on a separate server which will be accessed by other nodes. • You are permitted to use any pre-trained LLM, provided it is on a separate server, domain-specific, and context aware.

Demo Setup

For a good working demo, your distributed system should be designed to run on 4-5 nodes.

Suggested breakdown:

- **Node 1: AI/LLM Server** - Runs the lightweight, domain-specific LLM. Operates independently.
- **Node 2: Application Server with Raft Leader** - Manages core business logic and acts as Raft leader.
- **Node 3: Application Server with Raft Follower** - Replicates data for fault tolerance.

- **Node 4: Additional Raft Follower** - Redundancy for testing leader election and recovery.
- **Node 5: Client Node (multiple)** - Simulates multiple concurrent user interactions.

Function Signatures

The following are general expectations of core RPC functions your system should expose. Minor modifications are allowed, but RPC (via gRPC) must be used wherever remote communication is required.

Client Functions

`login(username, password)`

- Authenticates a user.
- Returns: `loginResponse(status, Optional(token))`

`logout(token)`

- Ends a user session.
- Returns: `status`

`post(token, type, data)`

- Sends new data (e.g., request, order, message, record update).
- Returns: `status`

`get(token, type, Optional(params))`

- Retrieves system data (e.g., seat availability, inventory levels, chat history, documents).
- Returns: `getResponse(e.g . status, List[(id, data)])`

Application Server Functions

`loginResponse(status, Optional(token))`

- Returns result of login request.

`getResponse(status, List[(id, data)])`

- Returns requested data items to the client.

processBusinessRequest(requestId, payload, context)

- Handles domain-specific operations (e.g., booking seat, editing document, updating record etc..).
- May interact with Raft for consistency and/or the LLM server for AI assistance.

LLM Server Functions

getLLMAnswer(requestId, query, context)

- Receives a query and context from the application server.
- Returns: getLLMAnswerResponse(requestId, answer)

Raft Consensus Functions

requestVote(from, to, term, lastLogIndex, lastLogTerm)

- Request vote for leadership.
- Returns: requestVoteReply(from, to, term, voteGranted)

requestVoteReply(from, to, term, voteGranted)

- Response to vote request.

appendEntries(from, to, term, prevIndex, prevTerm, commitIndex, entries)

- Sends new log entries for replication.
- Returns: appendEntriesReply(from, to, term, entryAppended, matchIndex)

appendEntriesReply(from, to, term, entryAppended, matchIndex)

- Acknowledges log replication and provides follower's log state.

Project Milestones and Deadlines (weightage 30%)

Milestone	Key Objectives	Deliverables
Milestone 1: Project Foundation and gRPC Deadline: Sept 28th, 2026 Hard Deadline: Sept 30th, 2026	• Team formation & topic selection • Basic gRPC service architecture • Fundamental business logic • Integrate domain LLM	• gRPC service definitions • Client-server communication • Authentication & sessions • LLM integration (sample queries) • Project structure & setup
Milestone 2: Raft Consensus Deadline: Nov 18th, 2026 Hard Deadline: Nov 20th, 2026	• Complete Raft consensus algorithm • Distributed data consistency • Fault tolerance & recovery • Stress testing	• Leader election mechanism • Log replication across nodes • Failure detection & recovery (demo consistency after leader node kill)

Viva will be conducted along with the final demonstration which shall be announced in due course of time.

Other Criteria & Submission Requirements

- **Plagiarism Detection**
- **Necessary Functionality**
- **Code Quality & Style**
- **Additional Features / Innovation**

Submission Requirements:

- **Code:** Compress to a zip file with readme file + code.
- **Drive link + video:** (5-10) minute demonstration of system functionality.
- **Documentation:** README with setup, deployment, and usage instructions.

- **Reflection Document:** Individual team member contributions and learnings.

Communication Channels

Questions or clarifications should be directed to the Course Instructor during designated office hours or through official communication channels.

Reference: "In Search of an Understandable Consensus Algorithm (Extended Version)" by Diego Ongaro and John Ousterhout.